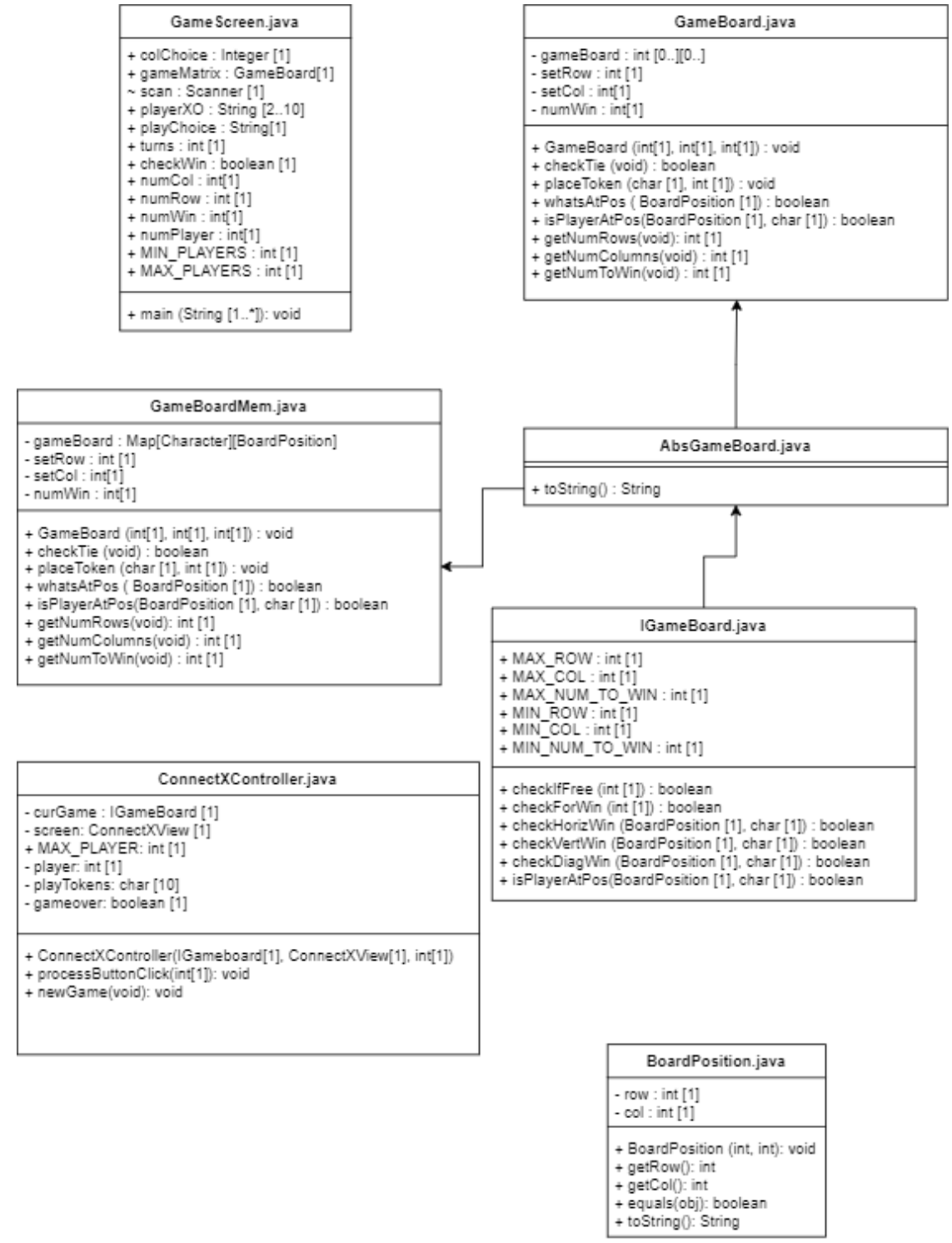
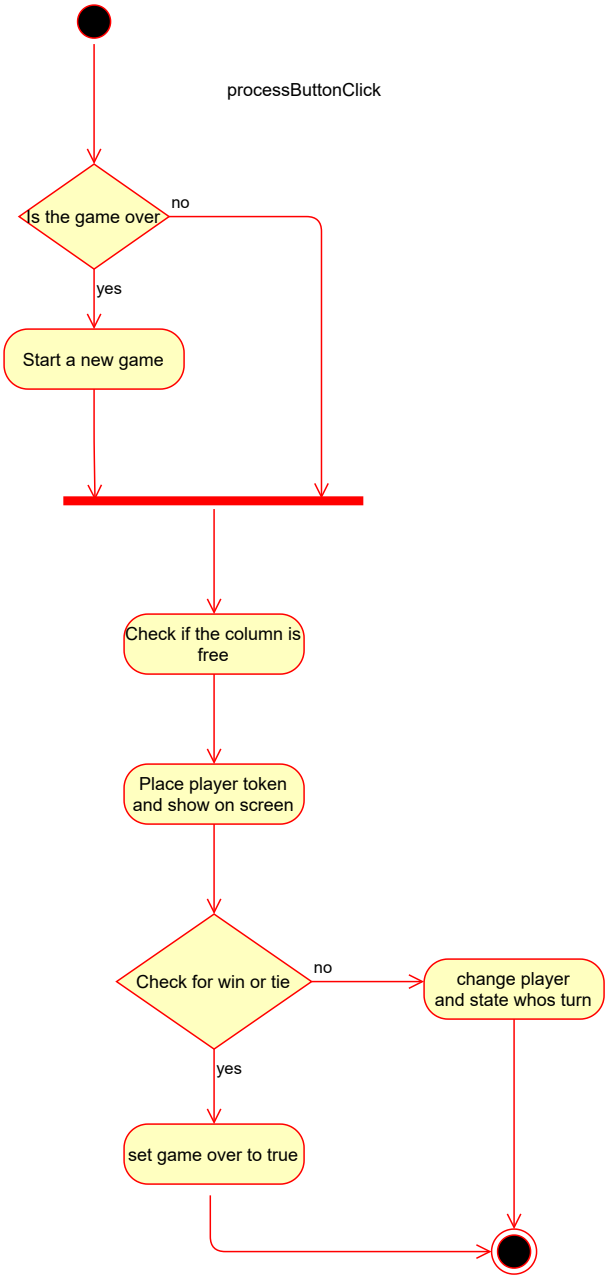
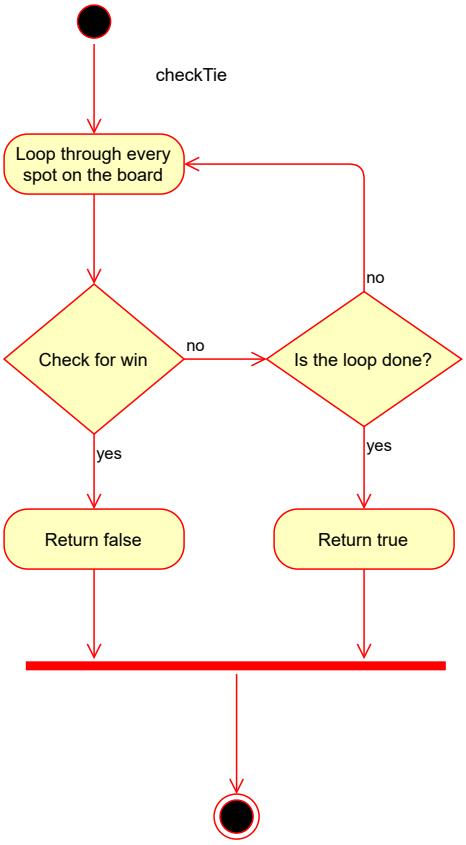
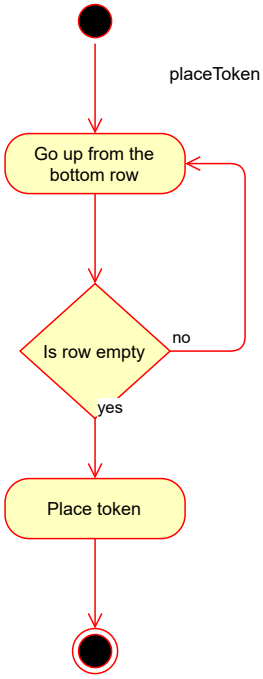
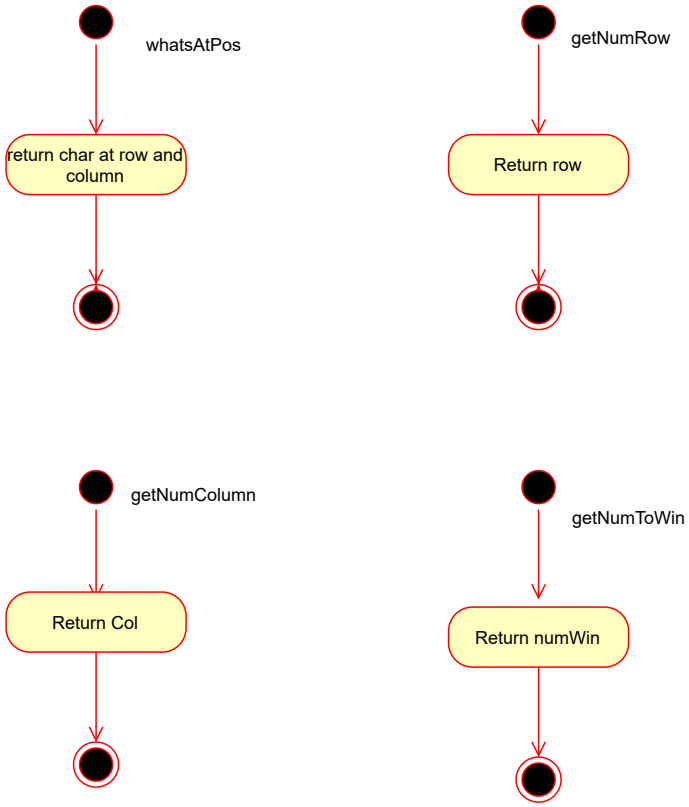
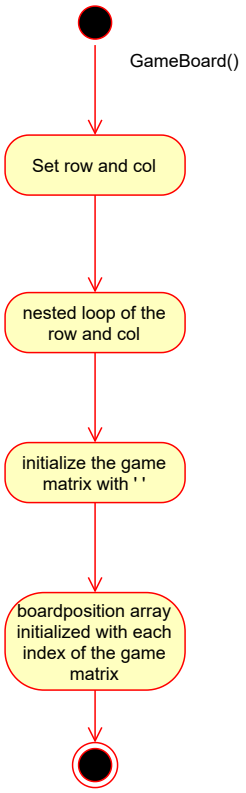

GameScreen.java
+ colChoice : Integer [1]
+ gameMatrix : GameBoard[1]
~ scan : Scanner [1]
+ playerXO : String [2..10]
+ playChoice : String[1]
+ turns : int [1]
+ checkWin : boolean [1]
+ numCol : int[1]
+ numRow : int [1]
+ numWin : int[1]
+ numPlayer : int[1]
+ MIN_PLAYERS : int [1]
+ MAX_PLAYERS : int [1]
+ main (String [1..*]): void
GameBoard.java
- gameBoard : int [0..][0..]
- setRow : int [1]
- setCol : int[1]
- numWin : int[1]
+ GameBoard (int[1], int[1], int[1]) : void
+ checkTie (void) : boolean
+ placeToken (char [1], int [1]) : void
+ whatsAtPos (BoardPosition [1]) : boolean
+ isPlayerAtPos(BoardPosition [1], char [1]) : boolean
+ getNumRows(void): int [1]
+ getNumColumns(void) : int [1]
+ getNumToWin(void) : int [1]
GameBoardMem.java
- gameBoard : Map[Character][BoardPosition]
- setRow : int [1]
- setCol : int[1]
- numWin : int[1]
+ GameBoard (int[1], int[1], int[1]) : void
+ checkTie (void) : boolean
+ placeToken (char [1], int [1]) : void
+ whatsAtPos (BoardPosition [1]) : boolean
+ isPlayerAtPos(BoardPosition [1], char [1]) : boolean
+ getNumRows(void): int [1]
+ getNumColumns(void) : int [1]
+ getNumToWin(void) : int [1]
AbsGameBoard.java
+ toString() : String
IGameBoard.java
+ MAX_ROW : int [1]
+ MAX_COL : int [1]
+ MAX_NUM_TO_WIN : int [1]
+ MIN_ROW : int [1]
+ MIN_COL : int [1]
+ MIN_NUM_TO_WIN : int [1]
+ checkIfFree (int [1]) : boolean
+ checkForWin (int [1]) : boolean
+ checkHorizWin (BoardPosition [1], char [1]) : boolean
+ checkVertWin (BoardPosition [1], char [1]) : boolean
+ checkDiagWin (BoardPosition [1], char [1]) : boolean
+ isPlayerAtPos(BoardPosition [1], char [1]) : boolean
ConnectXController.java
- curGame : IGameBoard [1]
- screen: ConnectXView [1]
+ MAX_PLAYER: int [1]
- player: int [1]
- playTokens: char [10]
- gameover: boolean [1]
+ ConnectXController(IGameboard[1], ConnectXView[1], int[1])
+ processButtonClick(int[1]): void
+ newGame(void): void
BoardPosition.java
- row : int [1]
- col : int [1]
+ BoardPosition (int, int): void
+ getRow(): int
+ getCol(): int
+ equals(obj): boolean
+ toString(): String

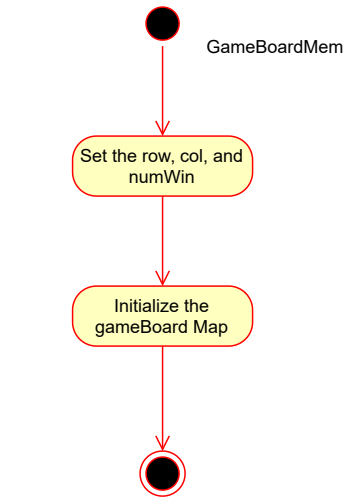

processButtonClick
Is the game over
no
yes
Start a new game
Check if the column is free
Place player token and show on screen
Check for win or tie
no
change player and state whos turn
yes
set game over to true

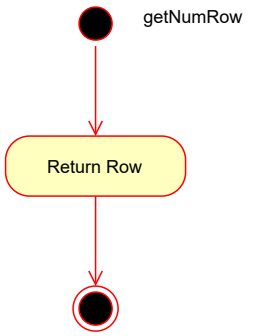

checkTie
Loop through every spot on the board
Check for win
no
Is the loop done?
no
yes
yes
Return false
Return true

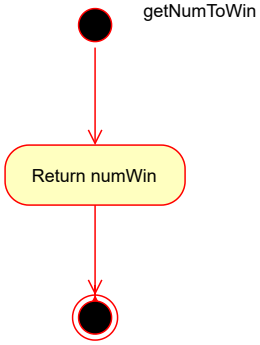

placeToken
Go up from the bottom row
Is row empty
no
yes
Place token

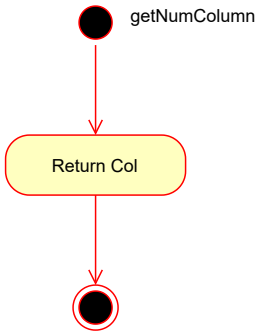

whatsAtPos
return char at row and column
getNumRow
Return row
getNumColumn
Return Col
getNumToWin
Return numWin

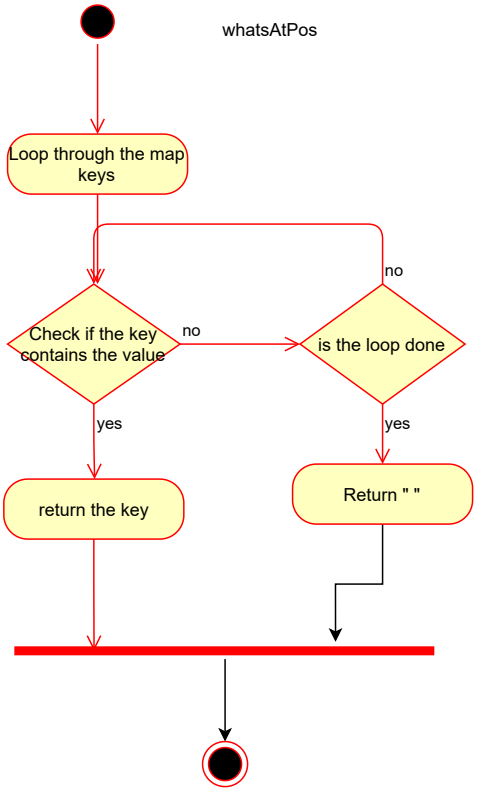

GameBoard()
Set row and col
nested loop of the row and col
initialize the game matrix with ' '
boardposition array initialized with each index of the game matrix

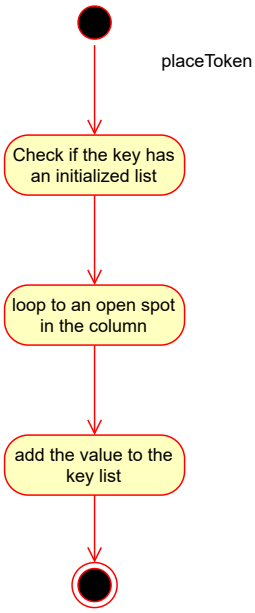

GameBoardMem
Set the row, col, and numWin
Initialize the gameBoard Map


getNumRow
Return Row

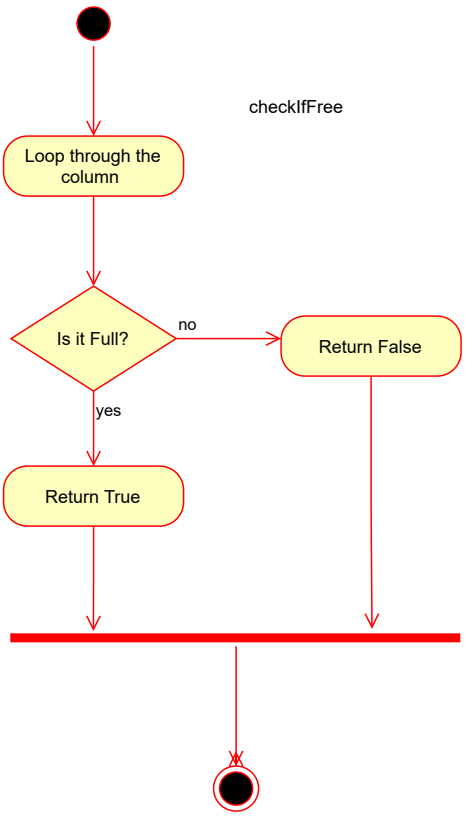

getNumToWin
Return numWin

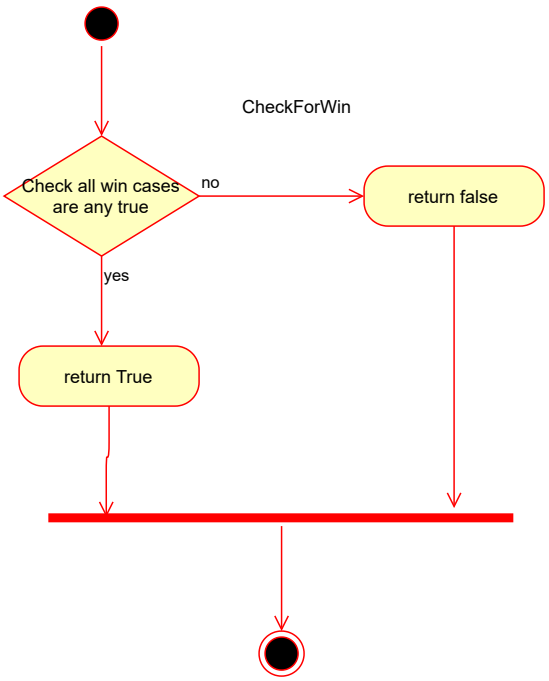

getNumColumn
Return Col


whatsAtPos
Loop through the map keys
Check if the key contains the value
no
is the loop done
no
yes
yes
return the key
Return " "

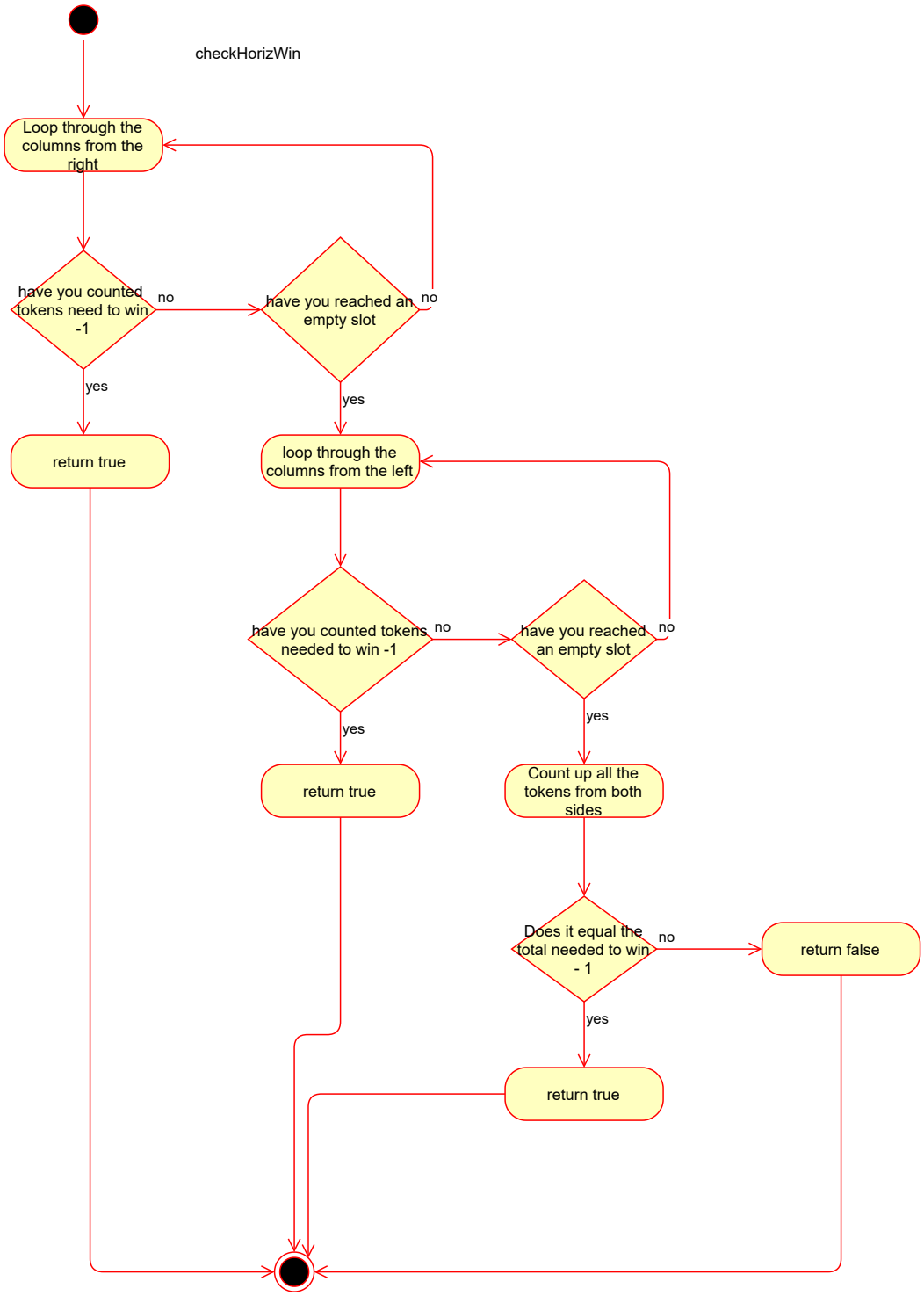

placeToken
Check if the key has an initialized list
loop to an open spot in the column
add the value to the key list

IGameboard

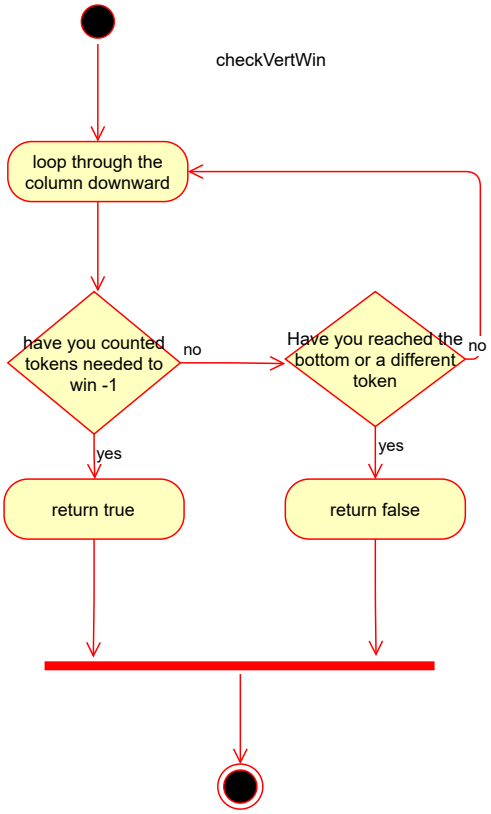

checkIfFree
Loop through the column
Is it Full?
no
Return False
yes
Return True

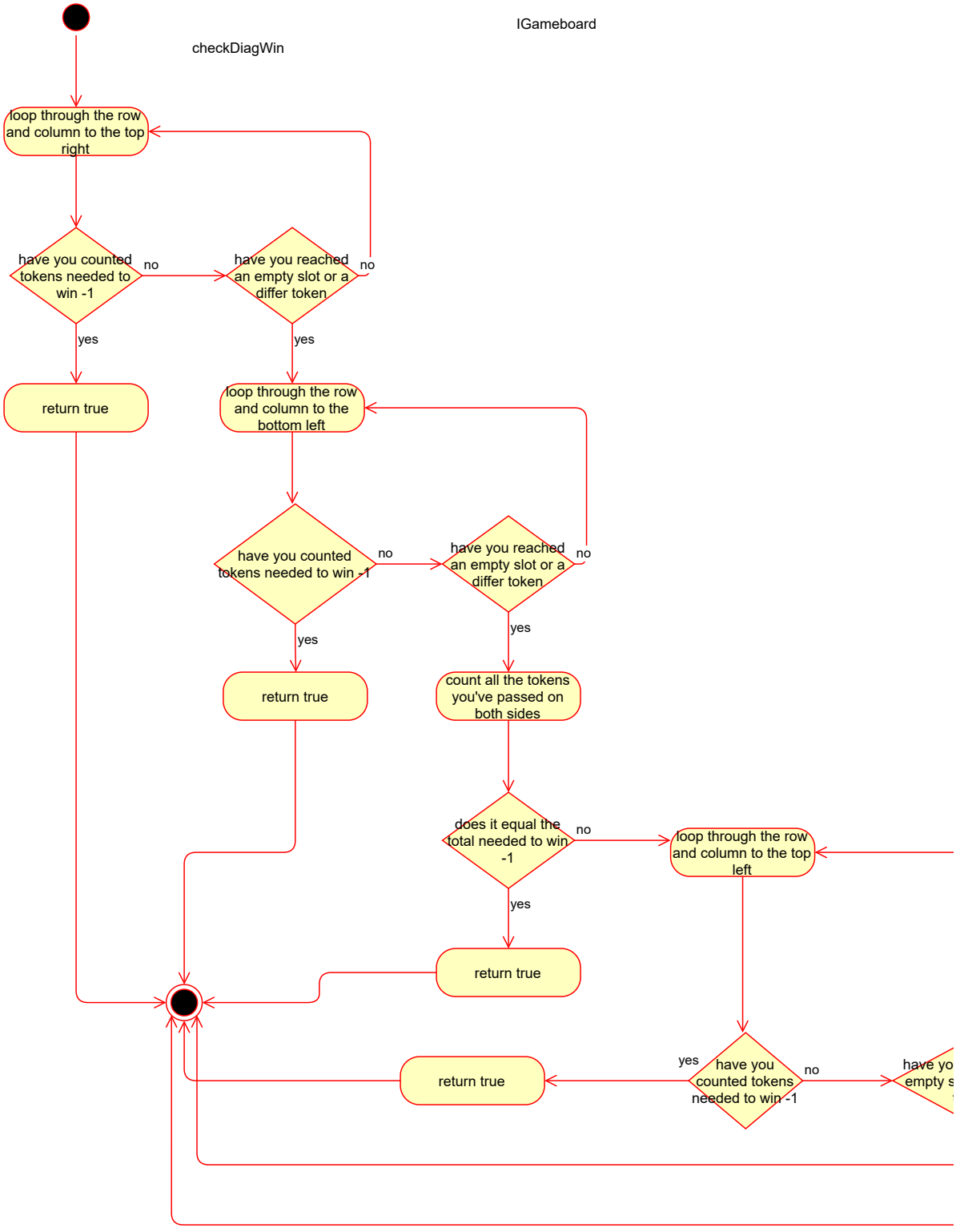

CheckForWin
Check all win cases are any true
no
return false
yes
return True

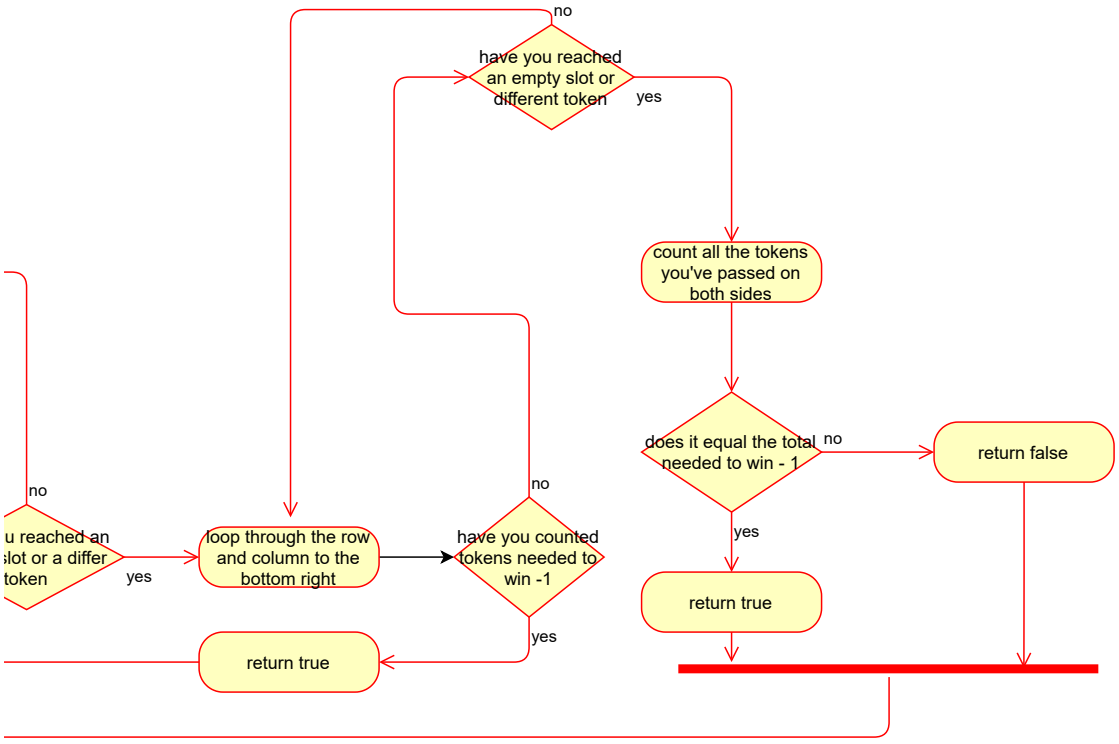
IGameboard

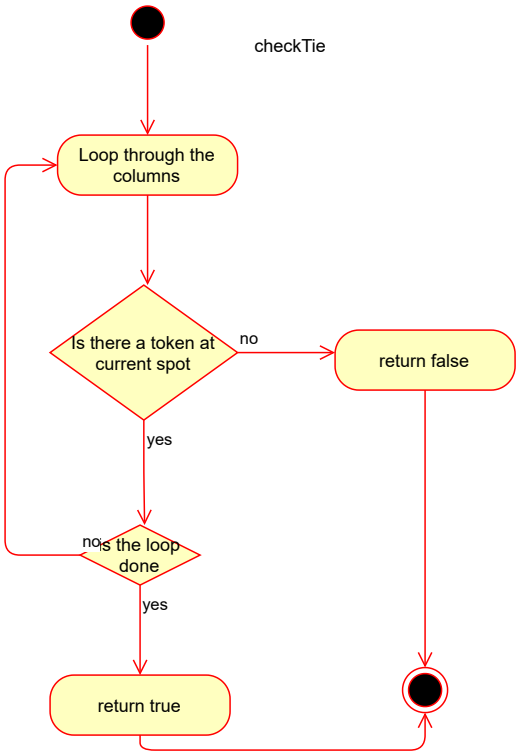

checkHorizWin
Loop through the columns from the right
have you counted tokens need to win -1
no
have you reached an empty slot
no
yes
yes
return true
loop through the columns from the left
have you counted tokens needed to win -1
no
have you reached an empty slot
no
yes
yes
return true
Count up all the tokens from both sides
Does it equal the total needed to win - 1
no
return false
yes
return true

IGameboard


checkVertWin
loop through the column downward
have you counted tokens needed to win -1
no
Have you reached the bottom or a different token
no
yes
yes
return true
return false


IGameboard
checkDiagWin
loop through the row and column to the top right
have you counted tokens needed to win -1
no
have you reached an empty slot or a differ token
no
yes
yes
return true
loop through the row and column to the bottom left
have you counted tokens needed to win -1
no
have you reached an empty slot or a differ token
no
yes
yes
return true
count all the tokens you've passed on both sides
does it equal the total needed to win -1
no
loop through the row and column to the top left
yes
return true
yes
have you counted tokens needed to win -1
no
return true


no
have you reached an empty slot or different token
yes
count all the tokens you've passed on both sides
does it equal the total needed to win - 1
no
return false
yes
return true
u reached an lot or a differ token
no
yes
loop through the row and column to the bottom right
have you counted tokens needed to win -1
no
yes
return true


checkTie
Loop through the columns
Is there a token at current spot
no
return false
yes
no
is the loop done
yes
return true

Requirements Analysis for Project 5

Functional Requirements:

- The user can place a chip on the gameboard to play.
- The user can play with up to 9 other people, so they can compete.
- The user can win by getting the set amount tokens in a row horizontally against the opponent.
- The user can win by getting the set amount tokens in a row vertically against the opponent.
- The user can win by getting the set amount tokens in a row diagonally against the opponent.
- The user can set the number of tokens needed to win the game.
- The user can set the number of columns on the board to play the game.
- The user can set the number of rows on the board to play the game.
- The user can not set the number to win to be more than both the number of columns and rows.
- The user can set the number of columns to be at most 100.
- The user can set the number of columns to be at least 3.
- The user can set the number of rows to be at most 100.
- The user can set the number of rows to be at least 3.
- The user can set the number of tokens in a row to win to be at most 25.
- The user can set the number of tokens in a row to win to be at least 3.
- The user can change the settings of the game when they rematch, allowing them to play with more or less people and larger or smaller boards.
- The user can view the printed gameboard to see open slots each turn.
- The user can restart the game to play again.
- The user can not place a chip out of bounds or in a full column
- The user can tie with the other players by taking all board spaces

Non-Functional Requirements:

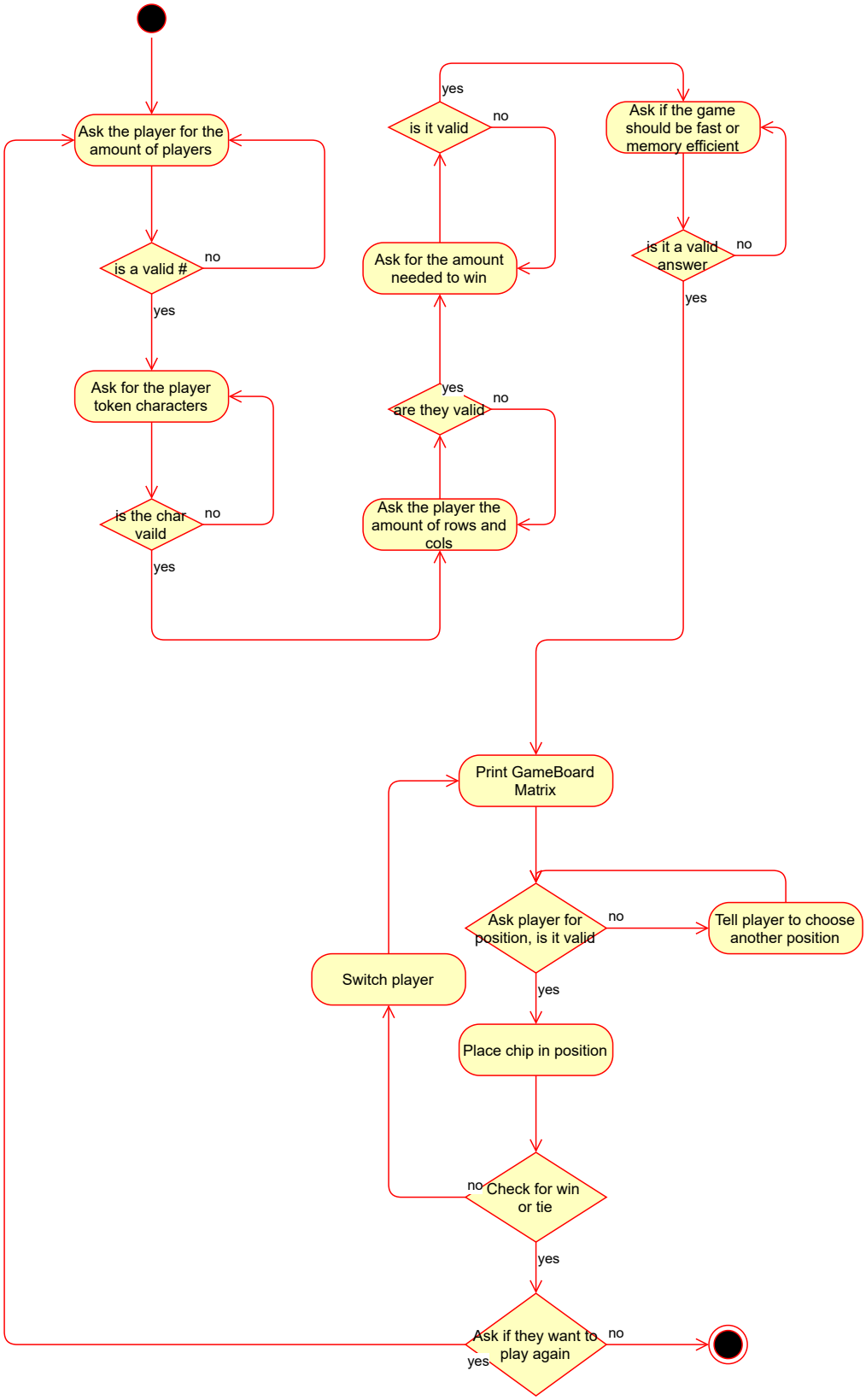
- The Application must run on Unix.
- The Application must compile on Unix.
- The Application must have a gameboard of at most 100x100.
- The Application must have a gameboard of at least 3x3.
- The Application must always start with the first player to choose their character.
- The Application must Alternate between all players.
- The Application must be able to restart after game is finished.

- The Application must have a Map variant of the gameboard.
- The application uses a fast or memory efficient version of the gameboard interchangeably
- The Application must have an Array variant of the gameboard.
- The Application must run on Windows.
- The Application must run on MacOS.
- The Application must be written in Java
- The Application gameboard bottom left corner must be (0,0)
- The Application uses a GUI
- The Application must have set characters for each player 1-10

Deployment

To run the code you must first be inside of the src folder. o Use the “make” command inside the terminal to compile all of the necessary files

- Use the “make run” command inside of the terminal to run the program’
- Use the “make test” command compile the test cases
- Use the “make testGB” to run the gameboard test cases
- Use the “make testGBMem” to run the gameboardMem test cases
- Use the “make clean” command inside of the terminal to the compiled (.class) files


Ask the player for the amount of players
is a valid #
no
yes
Ask for the player token characters
is the char vaild
no
yes
is it valid
yes
no
Ask for the amount needed to win
are they valid
yes
no
Ask the player the amount of rows and cols
Ask if the game should be fast or memory efficient
is it a valid answer
no
yes
Print GameBoard Matrix
Ask player for position, is it valid
no
Tell player to choose another position
yes
Place chip in position
Check for win or tie
no
yes
Switch player
Ask if they want to play again
yes
no

Test Cases Report

Constructor(int row, int col, int num_to_win)

Input	Output	Reason:
State: Object being created Row = 3 Col = 3 Num_to_win = 3	State: created Board Created	This test case input creates the minimum size board with the minimum amount of tokens needed to win. Function: testConstructor_min_values()

	0	1	2
2			
1			
0			

Constructor(int row, int col, int num_to_win)

Input	Output	Reason:
State: Object being created Row = 100 Col = 100 Num_to_win = 25	State: created Board Created	This test case input creates the maximum size board with the max amount of tokens needed to win. Function: testConstructor_max_values()

	0	1	…	99
99				
..				
1				
0				

Constructor(int row, int col, int num_to_win)

Input	Output	Reason:
State: Object being created Row = 3 Col = 4 Num_to_win = 3	State: created Board Created	This test case input creates the rectangular shape for the board when there's odd number or rows and column is even. While the number of tokens needed to win is the minimum. Function: testConstructor_rectangle_shape()

	0	1	2	3
2				
1				
0				

placeToken(char p, int c)

Input
State:

	0	1	2
2			
1			
0			

P = x
C = 1

Output
State: created

	0	1	2
2			
1			
0		X	

Reason:
This test case shows placing the token inside of a column that is completely empty.

Function:
testplaceToken_in_empty_column()

placeToken(char p, int c)

Input
State:

	0	1	2	99
99				
…				x
1				x
0				x

P = x
C = 99

Output
State: created

	0	1	…	99
99				X
…				X
1				X
0				X

Reason:
This test case shows placing a token in a column that has one space left before it's full.

Function:
testplaceToken_in_near_full_column()

placeToken(char p, int c)

Input
State:

	0	1	2	3	4
4					
3					
2					
1					
0					

P = x
C = 4

Output
State: created

	0	1	2	3	4
4					
3					
2					
1					
0					X

Reason:
This test case shows placing the token in the bottom right boundary of the board.

Function:
testplaceToken_bottom_right()

placeToken(char p, int c)

Input	Output	Reason:
State:	State: created	This test case shows placing the token in the bottom right boundary of the board.
4 | | |

checkIfFree(int c)

Input State:				Output State:				Reason:
	0	1	2		0	1	2	This test case shows checking a column that is not full and able to receive another token.
2				2				
1				1				Function: checkIfFree_column_available
0	X			0	X			
C = 0				checkIfFree = True				

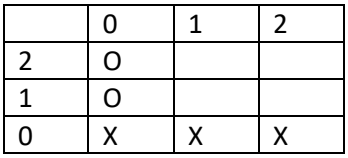
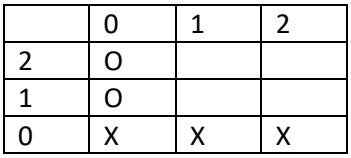
checkIfFree(int c)

Input State:				Output State:				Reason:
	0	1	2		0	1	2	This test case shows checking a column with one spot available to accept a token.
2	X	X		2	X	X		
1	X	X	X	1	X	X	X	Function: testcheckIfFree_last_column_only_available()
0	X	X	X	0	X	X	X	
C = 2								

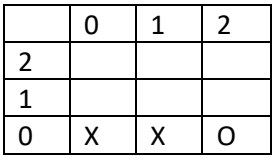
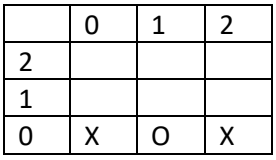
checkHorizWin(BoardPosition pos, int p)

Input State:				Output State: created				Reason:
	0	1	2		0	1	2	This test case shows checking if the last O placed, results in a win, showing that the function can identify the same tokens.
2				2				
1				1				Function: testCheckHorizWin_no_win()
0	X	O	X	0	X	O	X	
Pos = <0,1> P = O				checkHorizWin = false				

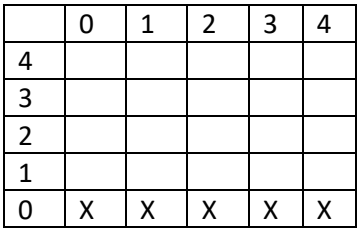
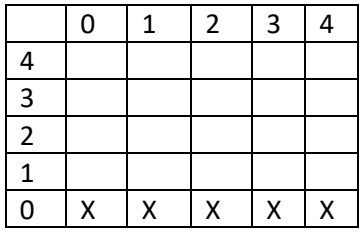
checkHorizWin(BoardPosition pos, int p)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the last X placed was in the lower right boundary of the board, so it has the function needs to detect it has reached the right boundary and go in the other direction.
		Function: testcheckHorizWin_bottom_right_win()
Pos = <0,2> P = X	checkHorizWin= false	

checkHorizWin(BoardPosition pos, int p)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the last X placed was in at the lower left boundary, so the function needs to recognize that it's reached the left boundary and start counting in the other direction.
		Function: testcheckHorizWin_lower_left_corner_placement(){
Pos = <0,0> P = X	checkHorizWin = false	

checkHorizWin(BoardPosition pos, int p)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the last X placed was in between two pieces, so the function needs to count X's on both the left and the right.
		Function: testcheckHorizWin_between_pieces()
Pos = <0,1> P = X	checkHorizWin = true	

checkVertWin(BoardPosition pos, int p)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the last O placed does not result in a vertical win because there are not any other similar tokens in either direction. Function: testcheckVertWin_no_win_below()

	0	1	2	3	4
4					
3					
2		O			
1		X			
0		X			

Pos = <2,1>
P = O

	0	1	2	3	4
4					
3					
2		O			
1		X			
0		X			

checkVertWin = true

checkVertWin(BoardPosition pos, int p)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the last X placed was in the highest row possible, so the function needs to notice it's reached the upper boundary and count X's below. Function: testcheckVertWin_top_row_add()

	0	1	2
2	X		
1	X		
0	X		

Pos = <2,0>
P = X

	0	1	2
2	X		
1	X		
0	X		

checkVertWin = true

checkVertWin(BoardPosition pos, int p)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the last X placed was in between an empty space and another piece. The winning string of pieces occurs in the middle of a column. Function: testcheckVertWin_middle_of_board_win()

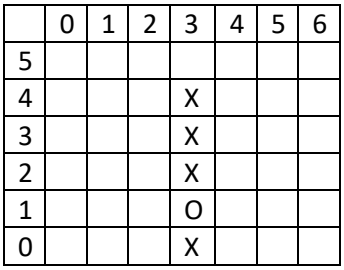
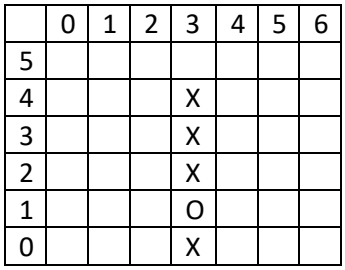
	0	1	2	3	4
4					
3				X	
2				X	
1				X	
0				O	

Pos = <3,3>
P = X

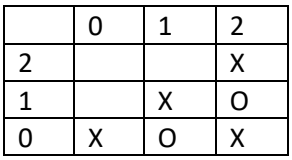
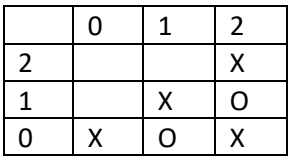
	0	1	2	3	4
4					
3				X	
2				X	
1				X	
0				O	

checkVertWin = true

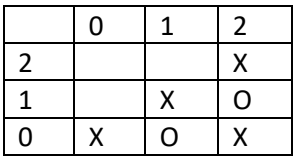
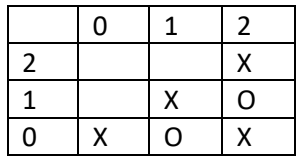
checkVertWin(BoardPosition pos, int p)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the last X placed was in a column that has the correct number of tokens needed to win (4) in the column but they are not in a row.
		Function: testcheckVertWin_right_number_nonconsecutive_token()
Pos = <4,3>	checkVertwin = false	
P = X		

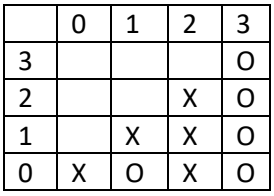
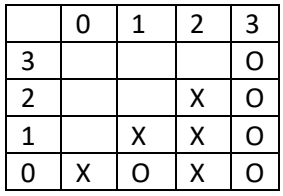
checkDiagWin(BoardPosition pos, int p)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the last X placed was in at the lower left boundary, so the function needs to recognize that it's reached the lower left bounds and start counting in the other direction.
		Function: testcheckDiagWin_bottom_left_placement_win()
Pos = <0,0>	checkDiagWin = true	
P = X		

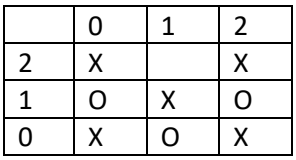
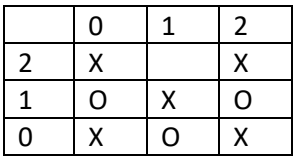
checkDiagWin(BoardPosition pos, int p)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the last X placed was in at the upper right boundary, so the function needs to recognize that it's reached the upper right bounds and start counting in the other direction.
		Function: testCheckDiagWin_upper_right_placement_win(
Pos = <2,2>	checkDiagWin = true	
P = X		

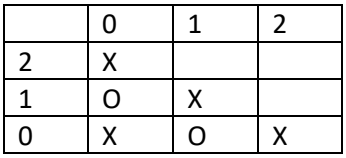
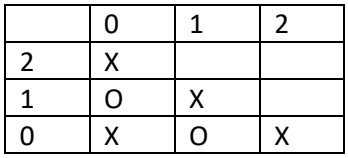
checkDiagWin(BoardPosition pos, int p)

Input State:	Output State: created	Reason:
 Pos = <2,2> P = X	 checkDiagWin = true	This test case is unique and distinct because the last X placed was diagonal to a token that's different, so the function needs to recognize this and start counting X's in the other direction. Function: testDiagWin_placed_diagonal_to_opponent_win(){

checkDiagWin(BoardPosition pos, int p)

Input State:	Output State: created	Reason:
 Pos = <1,1> P = X	 checkDiagWin = true	This test case is unique and distinct because the last X placed could results in a possible win from all directions. Function: testDiagWin_win_possible_from_all_directions()

checkDiagWin(BoardPosition pos, int p)

Input State:	Output State: created	Reason:
 Pos = <0,1> P = O	 checkDiagWin = true	This test case is unique and distinct because the last X placed was in at the lower right boundary, so the function needs to recognize that it's reached the lower right bounds and start counting in the other direction. Function: testDiagWin_bottom_right_placement()

checkDiagWin(BoardPosition pos, int p)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the last X placed was in at the upper left boundary, so the function needs to recognize that it's reached the upper left bounds and start counting in the other direction.
Pos = <2,0> P = X	checkDiagWin = true	Function: testDiagWin_upper_left_placement()

State:

	0	1	2
2	X		
1	O	X	
0	X	O	X

State: created

	0	1	2
2			
1			
0	X	O	X

checkDiagWin(BoardPosition pos, int p)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the last X placed was in a position to where it increase the tally by one in all directions but the function should reset the tally once the number to win(4) is not reach in each diagonal direction.
Pos = <2,2> P = X	checkDiagWin = false	Function: testDiagWin_tally_reset_in_all_directions()

State:

	0	1	2	3	4
4					
3		X		X	
2		X	X	X	
1		X	X	X	
0		X	X	X	

State: created

	0	1	2	3	4
4					
3		X		X	
2		X	X	X	
1		X	X	X	
0		X	X	X	

checkTie()

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because there is one position filled on this board.
	checkTie = false	Function: testcheckTie_one_piece_on_board()

State:

	0	1	2
2			
1			
0	X		

State: created

	0	1	2
2			
1			
0	X		

checkTie()

Input State:	Output State: created	Reason:
<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td>X</td><td>X</td><td>X</td></tr><tr><td>1</td><td>X</td><td>X</td><td>X</td></tr><tr><td>0</td><td>X</td><td>X</td><td>X</td></tr></table>	<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td>X</td><td>X</td><td>X</td></tr><tr><td>1</td><td>X</td><td>X</td><td>X</td></tr><tr><td>0</td><td>X</td><td>X</td><td>X</td></tr></table> checkTie = true	This test case is unique and distinct because all of the board is full, so this is a valid tie. This function needs to check every column. Function: testcheckTie_full_board()

checkTie()

Input State:	Output State of board unchanged checkTie = false	Reason: This test case is unique and distinct because all columns except the last one is full. So, the function has to check all the columns but stop on the last. Function: testcheckTie_all_but_one_top_space()
<table><tr><td></td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>4</td><td>X</td><td>X</td><td>X</td><td>X</td><td></td></tr><tr><td>3</td><td>X</td><td>X</td><td>X</td><td>X</td><td></td></tr><tr><td>2</td><td>X</td><td>X</td><td>X</td><td>X</td><td></td></tr><tr><td>1</td><td>X</td><td>X</td><td>X</td><td>X</td><td></td></tr><tr><td>0</td><td>X</td><td>X</td><td>X</td><td>X</td><td></td></tr></table>		

checkTie()

Input State:	Output State: created	Reason:
<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td>X</td><td></td><td></td></tr><tr><td>1</td><td>O</td><td></td><td></td></tr><tr><td>0</td><td>O</td><td></td><td></td></tr></table> Pos = <0,1> P = O	<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td>X</td><td></td><td></td></tr><tr><td>1</td><td>O</td><td></td><td></td></tr><tr><td>0</td><td>O</td><td></td><td></td></tr></table> checkTie = false	This test case is unique and distinct because the first column is full so the function will continue with checking but needs to abruptly stop at the next column. Function: testcheckTie_first_column_full()

isPlayerAtPos(BoardPosition pos, char player)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the player passed to the function is not at the position. So, the function will return false.
0 1 2 2 1 0 X	0 1 2 2 1 0 X	Function: testIsPlayerAtPos_wrong_char()
Pos = <0,0> Player = O	isPlayerAtPos = false	

isPlayerAtPos(BoardPosition pos, char player)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because the player is at bottom left position of the board.
0 1 2 2 1 0 X	0 1 2 2 1 0 X	Function: testIsPlayerAtPos_bottom_left_pos()
Pos = <0,0> Player = X	isPlayerAtPos = True	

isPlayerAtPos(BoardPosition pos, char player)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because it the player is at between two tokens in the middle of the lowest row.
0 1 2 2 1 0 X O X	0 1 2 2 1 0 X O X	Function: testIsPlayerAtPos_betweeen_tokens()
Pos = <0,1> Player = O	isPlayerAtPos = true	

isPlayerAtPos(BoardPosition pos, char player)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because it the player is at lower right boundary position of the board.
<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td>2</td><td></td><td>X</td><td>O</td></tr><tr><td>1</td><td></td><td>O</td><td>X</td></tr><tr><td>0</td><td>X</td><td>O</td><td>X</td></tr></table>	<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td></td><td></td></tr><tr><td>0</td><td>X</td><td>O</td><td>X</td></tr></table>	Function: testIsPlayerAtPos_bottom_right()
Pos = <0,2> Player = X	isPlayerAtPos = true	

isPlayerAtPos(BoardPosition pos, char player)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because it the player is at upper right boundary position of the board.
<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td></td><td></td><td>X</td></tr><tr><td>1</td><td></td><td></td><td>O</td></tr><tr><td>0</td><td></td><td></td><td>X</td></tr></table>	<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td></td><td></td><td>X</td></tr><tr><td>1</td><td></td><td></td><td>O</td></tr><tr><td>0</td><td></td><td></td><td>X</td></tr></table>	Function: testIsPlayerAtPos_upper_right_pos()
Pos = <2,2> Player = X		

whatsAtPos(BoardPosition pos)

Input	Output	Reason:
State:	State: created	This test case is unique and distinct because it looks at a position that doesn't contain a token. It returns a blank space character instead.
<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td></td><td></td></tr><tr><td>0</td><td></td><td></td><td></td></tr></table>	<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td></td><td></td></tr><tr><td>0</td><td></td><td></td><td></td></tr></table>	Function: testwhatsAtPos_empty_board()
Pos = <2,1>	whatsAtPos = ' '	

whatsAtPos(BoardPosition pos)

Input
State:

	0	1	2	3	4
4					
3					
2					
1		O	X		
0		O	X	X	

Pos = <1,1>

Output
State: created

	0	1	2
2			
1			
0	X	O	X

whatsAtPos = O

Reason:
This test case is unique and distinct because the it looks at a position that is in a middle row or column, not the bottom or top row.

Function:
testwhatsAtPos_middle_row()

whatsAtPos(BoardPosition pos)

Input
State:

	0	1	2
2		X	
1		O	
0		X	

Pos = <0,1>

Output
State: created

	0	1	2
2		X	
1		O	
0		X	

whatsAtPos = X

Reason:
This test case is unique and distinct because it looks at the token at a position that is adjacent to another token.
Function:
testwhatsAtPos_next_to_another_token()

whatsAtPos(BoardPosition pos)

Input State:	Output State: created	Reason:
<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td></td><td>X</td><td>O</td></tr><tr><td>1</td><td></td><td>X</td><td>O</td></tr><tr><td>0</td><td></td><td>O</td><td>X</td></tr></table> Pos = <2,2>	<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td></td><td>X</td><td>O</td></tr><tr><td>1</td><td></td><td>X</td><td>O</td></tr><tr><td>0</td><td></td><td>O</td><td>X</td></tr></table> whatsAtPos = O	This test case is unique and distinct because it looks at what's at the upper right boundary position of the board. Function: testwhatsAtPos_upper_right_bound()

whatsAtPos(BoardPosition pos)

Input State:	Output State: created	Reason:
<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td></td><td></td></tr><tr><td>0</td><td>X</td><td></td><td></td></tr></table> Pos = <0,0>	<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>2</td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td></td><td></td></tr><tr><td>0</td><td>X</td><td></td><td></td></tr></table> whatsAtPos = X	This test case is unique and distinct because it looks at what's at the lower left boundary position of the board. Function: testwhatsAtPos_lower_left_bound()